流程控制

程序的流程控制

- 顺序结构：代码自上而下，逐行执行
- 分支结构：根据条件，分情况执行
- 循环结构：根据条件，重复执行

导入模块

- Python 使用 import 导入非内置的模块。
- 常用的写法：
    - 将某个模块整个导入：import 模块 。导入后需要使用"模块名.函数名()"调用函数。
    - 将某个模块整个导入：import 模块 as 别名。导入后需要使用"模块别名.函数名()"调用函数。
    - 将某个模块的所有函数导入：from 模块 import *。导入后直接使用"函数名()"调用函数。
    - 将某个模块的某个函数导入：from 模块 import 函数。导入后直接使用"函数名()"调用函数。
    - 将某个模块的某些函数导入：from 模块 import 函数1, 函数2, ..., 函数n。导入后直接使用"函数名()"调用函数。
- 建议使用 import ... 方法导入，使用 from ... import ... 可能会导致函数名冲突。

生成随机数

- 导入 random 库。`import random`
- `random.randint(a,b)` 生成一个大于等于a且小于等于b的随机整数
- `random.randrange(a,b)` 生成一个大于等于a且小于b的随机整数
- `random.random()` 生成一个大于等于0且小于1之间的随机浮点数
- `random.uniform(a,b)` 生成一个a和b之间（包含a和b）的随机浮点数

例子：加法题

```
# AdditionQuiz.py
import random

# Generate random numbers
number1 = random.randint(0, 9)
number2 = random.randint(0, 9)

# Prompt the user to enter an answer
answer = eval(input("What is " + str(number1) + " + "
    + str(number2) + "? "))

# Display result
print(number1, "+", number2, "=", answer,
    "is", number1 + number2 == answer)
```

分支结构

- if语句。使用if语句可以进行条件判断，根据结果选择不同的运行分支。
- if语句的三种形式：if、if...else、if...elif...else
- 缩进：同一个代码段层次，需要相同的缩进，一般为4个空格，有些人喜欢用1个Tab。官方建议缩进使用4个空格。Python用缩进表达代码段的范围。
- pass语句：空语句，什么也不做，用于避免程序逻辑错误。

```
cond1 = cond2 = False
if cond1:
    pass

if cond1:
    pass
else:
    pass

if cond1:
    pass
elif cond2:
    pass
else:
    pass
```

例子：减法题

```
# SubtractionQuiz.py
import random

# 1. Generate two random single-digit integers
number1 = random.randint(0, 9)
number2 = random.randint(0, 9)

# 2. If number1 < number2, swap number1 with number2
if number1 < number2:
    number1, number2 = number2, number1 # Simultaneous assignment

# 4. Prompt the student to answer "what is number1 - number2?"
answer = eval(input("What is " + str(number1) + " - " +
    str(number2) + "? "))

# 4. Grade the answer and display the result
if number1 - number2 == answer:
    print("You are correct!")
else:
    print("Your answer is wrong.\n", number1, "-",
        number2, "is", number1 - number2)
```

例子：生肖年

```
# ChineseZodiac.py
year = eval(input("Enter a year: "))
zodiacYear = year % 12
if zodiacYear == 0:
    print("monkey")
elif zodiacYear == 1:
    print("rooster")
elif zodiacYear == 2:
    print("dog")
elif zodiacYear == 3:
    print("pig")
elif zodiacYear == 4:
    print("rat")
elif zodiacYear == 5:
    print("ox")
elif zodiacYear == 6:
    print("tiger")
elif zodiacYear == 7:
    print("rabbit")
elif zodiacYear == 8:
    print("dragon")
elif zodiacYear == 9:
    print("snake")
elif zodiacYear == 10:
    print("horse")
else:
    print("sheep")
```

例子：成绩等级

- 知识点：复合条件。如果想表示条件为：score大于等于80而且小于90

```
score >= 80 and score < 90      # 正确，且在别的语言，如C语言，也可以这样写
80 <= score and score < 90      # 正确，且在别的语言，如C语言，也可以这样写
80 <= score < 90                # Python中正确，但是在别的语言，如C语言中，是错误的
```

- if语句嵌套

```
score = eval(input("Enter the score: "))
if score >= 90:                 # 90 <= score and score <= 100
    grade = 'A'
else:
    if score >= 80:             # 80 <= score and score < 90
        grade = 'B'
    else:
        if score >= 70:         # 70 <= score and score < 80
            grade = 'C'
        else:
            if score >= 60:     # 60 <= score and score < 70
```

```
                grade = 'D'
            else:               # 0 <= score and score < 60
                grade = 'F'
print(grade)
```

- 多分支if语句

```
score = eval(input("Enter the score: "))
if score >= 90:     # 90 <= score <= 100
    grade = 'A'
elif score >= 80:   # 80 <= score < 90
    grade = 'B'
elif score >= 70:   # 70 <= score < 80
    grade = 'C'
elif score >= 60:   # 60 <= score < 70
    grade = 'D'
else:               # 0 <= score < 60
    grade = 'F'
print(grade)
```

- 多分支if语句的错误示例

```
# 以下为错误代码!
score = eval(input("Enter the score: "))
if score >= 60:
    grade = 'D'
elif score >= 70:
    grade = 'C'
elif score >= 80:
    grade = 'B'
elif score >= 90:
    grade = 'A'
else:
    grade = 'F'
print(grade)
```

例子：计算 BMI 并给出建议

```
# ComputeBMI.py
# Prompt the user to enter weight in kilograms
weight = eval(input("Enter weight in kilograms: "))

# Prompt the user to enter height in meters
height = eval(input("Enter height in meters: "))

# Compute BMI
bmi = weight / (height * height)

# Display result
print("BMI is", format(bmi, ".2f"))
if bmi < 18.5:
```

```
    print("Underweight")
elif bmi < 25:
    print("Normal")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

例子：判断某年是否为闰年

- 现行为主流历法为格里高利历，其置闰规则为：
    - 一般情况（非世纪年），如果年份数除以4可以除得尽，则置闰，即2月有29天，全年有366天。
    - 特殊情况（世纪年：即最后两位为00的年份），如果年份数除以400可以除得尽，则置闰，即2月有29天，全年有366天。
    - 其它情况，为平年，不置闰，即2月有28天，全年有365天
- 为什么要置闰?
    - 经天文测量，目前一回归年约为365.2422天。
    - 如果不置闰年，则一年有365天，误差为0.2422天/年。
    - 如果不处理世纪年，则一年有365.25天，误差为0.0078天/年。
    - 如果处理世纪年，则一年有365.2425天，误差为0.0003天/年。

```
# LeapYear.py
year = eval(input("Enter a year: "))

# Check if the year is a leap year
# method 1
isLeapYear = False
if year % 100 == 0:
    if year % 400 == 0:
        isLeapYear = True
else:
    if year % 4 == 0:
        isLeapYear = True

# method 2
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

# Display the result
print(year, "is a leap year?", isLeapYear)
```

- 小知识：历史上消失的11天

例子：求一个月的天数

- 如果想判断月份 month 是小月

```
month == 4 or 6 or 9 or 11 # 错误！6意为True
month == 4 or month == 6 or month == 9 or month == 11 # 正确，代码较长
month in (4,6,9,11) # 正确，使用元组更简练
```

```
year = eval(input())
month = eval(input())

if month == 2:
    if year % 400 == 0 or year % 100 !=0 and year % 4 == 0:
        days = 29  # 闰年，2月为29天
    else:
        days = 28  # 平年，2月为28天

elif month in (1,3,5,7,8,10,12):
    days = 31

elif month == 4 or month == 6 or month == 9 or month == 11:
    days = 30

print(days)
```

条件表达式

- 类似于C语言中的三元运算符
- 格式为：表达式1 if 条件 else 表达式2。当条件成立时，结果为表达式1，否则，结果为表达式2

```
x = eval(input())
if x>=0:
    y=1
else:
    y=-1

# 等价的条件表达式

y = 1 if x>=0 else -1
```

```
number = eval(input())

# 判断 number 是否为偶数
# 方法1
if number % 2 == 0:
    even = True
else:
    even = False
if even:
```

```
    print("number is even")
else:
    print("number is odd")

# 方法2
even = number % 2 == 0
if even:
    print("number is even")
else:
    print("number is odd")

# 方法3
print("number is even" if number % 2 == 0 else "number is odd")
```

例子：随机生成一个四则运算题

```
import random

n1 = random.randint(1,9)
n2 = random.randint(1,9)

method = random.randint(1,4)

if method == 1: # 出加法题
     print(n1, "+", n2, "=?")
     answer = int(input())
     if answer == n1+n2:
           print("回答正确")
     else:
           print("回答错误，正确答案是", n1 + n2)
elif method == 2: # 出减法题
     if n1 < n2:
          n1,n2 = n2, n1
     print(n1, "-", n2, "=?")
     answer = int(input())
     if answer == n1-n2:
           print("回答正确")
     else:
           print("回答错误，正确答案是", n1 - n2)
elif method == 3:  # 出乘法题
     print(n1, "*", n2, "=?")
     answer = int(input())
     if answer == n1*n2:
           print("回答正确")
     else:
           print("回答错误，正确答案是", n1 * n2)
else:  # 出除法题
     n3 = n1 * n2
     print(n3, "/", n1, "=?")
     answer = int(input())
     if answer == n2:
           print("回答正确")
```

```
    else:
        print("回答错误，正确答案是", n2)
```

循环结构

while语句

- 遇到while语句时，如果条件成立，while执行循环体中的语句
- 执行完每一轮循环后，while会重新判断条件，如果条件仍然成立，则再次执行循环体中的语句

```
cond1 = True
while cond1:
    pass
```

例子：从1累加到100求和

```
i=1
sum=0
while i<=100:
    sum += i
    i += 1
print(sum)
```

- 在执行到while语句时，条件如果为False，则直接跳过循环，执行循环之后的语句。
- 在循环体中，必须有使条件判断结果发生变化的语句，否则循环将永远执行下去（称为死循环）。

```
x = eval(input())
while x!= 0:
    print(x)
    x -= 3
print("Finish.")
```

例子：猜数字

```
# GuessNumber.py
import random

# Generate a random number to be guessed
number = random.randint(1, 100)

print("Guess a magic number between 1 and 100")

guess = -1
while guess != number:
    # Prompt the user to guess the number
    guess = eval(input("Enter your guess: "))

    if guess == number:
```

```
        print("Yes, the number is", number)
    elif guess > number:
        print("Your guess is too high")
    else:
        print("Your guess is too low")
```

for...in语句

- for...in语句一般用于实现已经循环次数的循环。
- 基本结构：for 变量 in 序列，循环执行的次数与序列中元素的个数相同，每次循环中，变量自动赋值为序列中的某个元素。

```
for c in 'hello':   # 循环5次，依次输出 h, e, l, l, o 五个字母
    print(c)

for i in (2,3,5,7): # 循环4次，依次输出 2, 3, 5, 7 四个数字
    print(i)
```

range()函数

- range()函数经常用在for...in语句中
- range(n)：产生一个 0,1,2,3,....,n-1 这n个数的序列，注意是从0开始，总数为n个。
    - 如果 n<= 0，生成的序列为空。
    - 例：range(5)：产生 0,1,2,3,4 这 5 个数组成的序列
- range(a,b)：产生从 a 到 b 的数字序列，注意是大于等于a且小于b，即包含 a 而不包含 b！
    - 如果b<=a，生成的序列为空。
    - 例：range(1,10)：产生 1,2,3,...,9 这 9 个数的序列
- range(a,b,k)函数：产生一个a,a+k,a+k+k,...序列，即从 a 开始向 b 行进，但不能达到 b，即：
    - 如果 k>0，生成序列中的最大值不能大于或等于b。如果b<=a，生成的序列为空。
    - 如果 k<0，生成序列中的最大值不能小于或等于b。如果b>=a，生成的序列为空。
    - k不能等于0。省略参数 k，相当于 k=1
    - 例：range(0,10,2)，将产生0,2,4,6,8这个序列
    - 例：range(10,1,-1)，将产生10,9,8,7,6,5,4,3,2这个序列

例题：二个年份之间多少天

- 输入一个起始年份和一个终止年份，计算并输出一共有多少天?

```
year1 = int(input())
year2 = int(input())

days = 0
for i in range(year1,year2+1):
    if i%400==0 or i%100!=0 and i%4==0:
        days += 366
    else:
        days += 365

print(days)
```

while和for循环的转换

- 一般来说，已知循环次数的情况，可以用for循环，也可以用while循环，且for循环往往比较简单。
- 未知循环次数的情况，一般使用while循环。

```
# 当已知循环次数时，可以使用以下两种循环方式

# while 循环
i = 1
while i < n:
    pass
    i += 1

# for 循环
for i in range(1,n):
    pass
```

不要比较浮点数

- 例如：计算 1+0.9+0.8.....+0.2+0.1

```
item = 1
sum = 0

while item != 0:     # 死循环，无法退出
    sum += item
    item -= 0.1
    print(item)

print(sum)
```

- 以下是循环中 item 的值

```
0.9
0.8
0.7000000000000001
0.6000000000000001
```

```
0.5000000000000001
0.40000000000000013
0.30000000000000016
0.20000000000000015
0.10000000000000014
1.3877787807814457e-16
-0.09999999999999987
-0.19999999999999987
-0.2999999999999999
-0.3999999999999999
-0.4999999999999999
-0.5999999999999999
-0.6999999999999998
-0.7999999999999998
-0.8999999999999998
-0.9999999999999998
-1.0999999999999999
-1.2
-1.3
-1.4000000000000001
-1.5000000000000002
-1.6000000000000003
-1.7000000000000004
-1.8000000000000005
-1.9000000000000006
-2.0000000000000004
.....................
```

- 那，不用等于来判断，改成小于等于行不行？再例：求和。0.01 + 0.02 + ... + 0.99 + 1

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
i = 0.01
while i <= 1.0:
    sum += i
    i = i + 0.01

# Display result
print("The sum is", sum)

# 输出： The sum is 49.50000000000003
```

- 预期得到50.5，但是.....结果是49.5 。为什么呢？看一下循环变量 i 的值：

```
0.01
0.02
0.03
0.04
0.05
0.060000000000000005
```

```
0.07
0.08
0.09
0.09999999999999999
0.10999999999999999
0.11999999999999998
0.12999999999999998
0.13999999999999999
0.15
.....
0.9700000000000006
0.9800000000000006
0.9900000000000007
1.0000000000000007
```

- 最后一个值是1.0000000000000007，本来这个值也应该加入到sum值中，但是因为它已经大于1了，最后一轮循环并未执行，所以结果差了1 。
- 修改方法：将循环次数的控制变量改为整数。

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
i = 0.01
for count in range(100):
    sum += i
    i = i + 0.01

# Display result
print("The sum is", sum)

# 输出： The sum is 50.50000000000003
```

例子：判断是否为回文数

- 回文数，即倒过来读与原数字相同的数，例如：12321

```
number = int(input('请输入一个整数：'))
n = number
m = 0

# 从原数中从个位开始每次向高1位取一个数，然后让生成的新数*10再加上从原数中取出的这个数，重复以上动作，逐渐把原数中的每一位颠倒过来
while n !=0:
    m = m * 10 + n % 10
    n //= 10

if m == number:
    print(number,'是回文数')
else:
    print(number,'不是回文数')
```

例子：打印九九乘法表

- print()中加上end参数，可以输出后不换行，而是输出end的值。

```
for i in range(1,10):
    for j in range(1,10):
        k = i * j
        if k < 10:
            print(" ",end="") #如果是一位数，那么在前面加个空格，且不换行
        print(k,end=" ") # 输出一个数字，且不换行
    print() # 仅换行
```

- 带格式版

```
# MultiplicationTable.py
print("          Multiplication Table")
# Display the number title
print("  |", end = '')
for j in range(1, 10):
    print("  ", j, end = '')
print() # Jump to the new line
print("-----------------------------------------")

# Display table body
for i in range(1, 10):
    print(i, "|", end = '')
    for j in range(1, 10):
        # Display the product and align properly
        print(format(i * j, '4d'), end = '')
    print()# Jump to the new line
```

例子：打印金字塔

- 输入一个不大于9的正整数n，输出一个n行金字塔，如下所示

```
            1
          1 2 1
        1 2 3 2 1
      1 2 3 4 3 2 1
    1 2 3 4 5 4 3 2 1
  1 2 3 4 5 6 5 4 3 2 1
1 2 3 4 5 6 7 6 5 4 3 2 1
```

```
n = int(input())
for i in range(1,n+1):
    for j in range((n-i)*2):
        print(" ",end="")
    for j in range(1,i+1):
        print(j,end=' ')
    for j in range(i-1,0,-1):
        print(j,end=' ')
    print()
```

break 和 continue

- break语句可以跳出 for 和 while 的循环体
- 例：循环输入一个数字，输出其平方，直到输入0为止。

```
while True:
    x = int(input())
    if x == 0:
        break
    print(x**2)
```

- continue语句跳出当前轮循环，直接进行下一轮循环
- 例：打印100以内的正整数中，不可以被7整除，数字中也不含7的整数。

```
for i in range(1,101):
    if i % 7 == 0:
        continue
    if i % 10 == 7:
        continue
    if i // 10 == 7:
        continue
    print(i)
```

例子：求二个正整数的最大公约数

```
#Prompt the user to enter two integers
n1 = eval(input("Enter first integer: "))
n2 = eval(input("Enter second integer: "))

gcd = 1
k = 2
while k <= n1 and k <= n2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1

print("The greatest common divisor for",
    n1, "and", n2, "is", gcd)
```

- 优化：从大往小找，找到的第一个符合条件的就是最大公约数

```
a = int(input())
b = int(input())
if a < b:
    a,b = b,a
for i in range(b,0,-1):
    if a % i == 0 and b % i == 0:
        break
print(i)
```

- 辗转相除法（又称**欧几里得算法**）求最大公约数

```
# 辗转相除法
a = int(input())
b = int(input())
if a < b:
    a,b = b,a
c = a % b
while c!=0:
    a = b
    b = c
    c = a % b
print(b)
```

循环中的else

- else如果与while或for语句配合使用，意思是当循环正常退出（即不是遇到break语句而退出）时，就会执行else子句。
- 此语法为Python特有，其它语言（如C语言）一般没有将else用于循环语句中的语法。

例子：判断一个数是否为素数

```
number = int(input())
isPrime = True
for i in range(2,number):
    if number % i == 0:
        isPrime = False
        break
if isPrime:
    print(number,"is a prime number")
else:
    print(number,"is not a prime number")
```

- 循环中使用else，如果不执行break，则执行else

```
number = int(input())
for i in range(2,number):
    if number % i == 0:
        break
else:
    print(number,"is a prime number")
```

- 优化：其实循环结束的上限，不必达到 number-1，而是达到 math.ceil(number**0.5) 即可。原因：如果 n 是合数，则 n 必存在一个因数a小于等于n的平方根。证明如下：
    - 设 m = math.ceil(n**0.5)，如果存在 a,b 且 a<=b，使得 a*b = n，那么必然 a<=m。
    - 反证法：假设a>m。因为a<=b，所以b>m，所以 a*b > m**2 ，已知 n=a*b，代入上式，n > m**2，与m的定义矛盾。

```
number = int(input())
isPrime = True
for i in range(2,math.ceil(number**0.5)+1):
    if number % i == 0:
        isPrime = False
        break
if isPrime:
    print(number,"is a prime number")
else:
    print(number,"is not a prime number")
```

异常处理

异常（exception）

- python运行出错会停止并报错 - - 引发异常。
- 通过异常处理机制，可以检查并处理异常，以便使程序不至于突然结束。

```
>>> num=float('hi')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: 'hi'
```

使用try语句及except子句

- try用于将可能引发异常的代码隔开
- except用于标记当异常出现时需要执行的代码

```
# Handle It
# Demonstrates handling exceptions
# try/except
try:
    num = float(input("Enter a number: "))
except:
    print("Something went wrong!")
```

指定异常类型

- 不同错误会引发不同类型的异常
- except子句可以指定类型
- 指定类型是好习惯
- 有和用户交互的地方通常容易出错

```
# specifying exception type
try:
    num = float(input("\nEnter a number: "))
except ValueError:
    print("That was not a number!")
```

- 不知道类型名时可以这样
- 就知道除0异常名字了

```
>>> print(1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

异常类型

异常名称	描述
BaseException	所有异常的基类
SystemExit	解释器请求退出
KeyboardInterrupt	用户中断执行(通常是输入^C)
Exception	常规错误的基类
StopIteration	迭代器没有更多的值
GeneratorExit	生成器(generator)发生异常来通知退出
SystemExit	Python 解释器请求退出
StandardError	所有的内建标准异常的基类
ArithmeticError	所有数值计算错误的基类

异常名称	描述
FloatingPointError	浮点计算错误
OverflowError	数值运算超出最大限制
ZeroDivisionError	除(或取模)零 (所有数据类型)
AssertionError	断言语句失败
AttributeError	对象没有这个属性
EOFError	没有内建输入,到达EOF 标记
EnvironmentError	操作系统错误的基类
IOError	输入/输出操作失败
OSError	操作系统错误
WindowsError	系统调用失败
ImportError	导入模块/对象失败
KeyboardInterrupt	用户中断执行(通常是输入^C)
LookupError	无效数据查询的基类
IndexError	序列中没有没有此索引(index)【越界】
KeyError	映射中没有这个键
MemoryError	内存溢出错误(对于Python 解释器不是致命的)
NameError	未声明/初始化对象 (没有属性)
UnboundLocalError	访问未初始化的本地变量
ReferenceError	弱引用(Weak reference)试图访问已经垃圾回收了的对象
RuntimeError	一般的运行时错误
NotImplementedError	尚未实现的方法
SyntaxError	Python 语法错误
IndentationError	缩进错误
TabError	Tab 和空格混用
SystemError	一般的解释器系统错误
TypeError	对类型无效的操作
ValueError	传入无效的参数
UnicodeError	Unicode 相关的错误

异常名称	描述
UnicodeDecodeError	Unicode 解码时的错误
UnicodeEncodeError	Unicode 编码时错误
UnicodeTranslateError	Unicode 转换时错误
Warning	警告的基类
DeprecationWarning	关于被弃用的特征的警告
FutureWarning	关于构造将来语义会有改变的警告
OverflowWarning	旧的关于自动提升为长整型(long)的警告
PendingDeprecationWarning	关于特性将会被废弃的警告
RuntimeWarning	可疑的运行时行为(runtime behavior)的警告
SyntaxWarning	可疑的语法的警告
UserWarning	用户代码生成的警告

处理多种异常

- 一个try语句可以捕获多种异常
- 方法一

```
# handle multiple exception types
for value in (None, "Hi!"):
try:
    print("Attempting to convert", value, "-->", end=" ")
    print(float(value))
except (TypeError, ValueError):
    print("Something went wrong!")
```

- 方法二，这样可以有针对性的给出解决办法

```
for value in (None, "Hi!"):
try:
    print("Attempting to convert", value, "-->", end=" ")
    print(float(value))
except TypeError:
    print("I can only convert a string or a number!")
except ValueError:
    print("I can only convert a string of digits!")
```

- 对于无法确定的异常，在最后给出默认处理

```
for value in (None, "Hi!"):
try:
    print("Attempting to convert", value, "-->", end=" ")
    print(float(value))
except TypeError:
    print("I can only convert a string or a number!")
except ValueError:
    print("I can only convert a string of digits!")
except:
    print("Something is wrong, but I don't know why.")
```

获取异常的参数

- 异常发生时可能会有与之关联的值—异常的参数
- 是python描述异常的正式消息，可以这样得到

```
# get an exception's argument
try:
    num = float(input("\nEnter a number: "))
except TypeError as e:
    print("I can only convert a string or a number! See: ", e)
except ValueError as e:
    print("That was not a number! Or as Python would say...", e)
except Exception as e:
    print("Something is wrong. As Python would say...", e)
```

else子句

- 在try语句最后加上else子句代表没有异常时要做的事

```
# try/except/else
try:
    num = float(input("\nEnter a number: "))
except TypeError as e:
    print("I can only convert a string or a number! See: ", e)
except ValueError as e:
    print("That was not a number! Or as Python would say...", e)
except Exception as e:
    print("Something is wrong. As Python would say...", e)
else:
    print("You entered the number", num)
```

finally子句

- 在try语句最后加上finally子句代表无论如何要做的事

```
try:
    num = float(input("\nEnter a number: "))
except TypeError as e:
    print("I can only convert a string or a number! See: ", e)
except ValueError as e:
    print("That was not a number! Or as Python would say...", e)
except Exception as e:
    print("Something is wrong. As Python would say...", e)
else:
    print("You entered the number", num)
finally:
    print("I'm always here!")
```

抛出异常 raise语句

- raise语句可以主动抛出异常

```
>>> raise Exception("This is the error message")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
Exception: This is the error message
```

```
try:
    number = float(input("Enter a positive number: "))
    if number <= 0:
        raise ValueError("Not a positive number!")
except Exception as e:
    print("Something is wrong. ", e)
```

调试程序

断言（assert）

- 断言：assert 条件(True/False), “条件为False时显示的提示信息”。
- 假如你预期此处x的值为0.3，如果不是0.3，则抛出异常。

```
>>> x=0.1+0.2
>>> assert x==0.3, "Ooop, x is wrong"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError: Ooop, x is wrong
```

- 断言针对的是程序员的错误，而不是用户的错误。
- 对于可以恢复的错误，请抛出异常，而不是用 assert 语句检测它。
- 运行python时使用 -O 选项，可以禁用断言。

日志（logging）

- 使用日志 logging 记录程序的运行过程和状态
- 不要用print()调试。会与正常的输出混淆。
- 日志是给程序员看的，不是给用户看的。

日志级别和日志函数

日志级别	日志函数	描述
DEBUG	logging.debug()	调试，最低级别，调试诊断时的问题
INFO	logging.info()	信息，一般事件的信息，或确认一切正常
WARNING	logging.warning()	警告，表示可能存在问题，不会阻止程序的工作
ERROR	logging.error()	错误，会导致程序失败
CRITICAL	logging.critical()	严重，最高级别，表示致命错误，会导致程序完全停止

```
import logging
logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(levelname)s - %(message)s')
x = 1
logging.debug("x="+str(x))
# 2023-03-09 15:28:14,204 - DEBUG - x=1
```

- 禁用日志： logging.disable(日志级别)，等于或低于此级别的日志禁用
- 禁用所有日志：logging.disable(logging.CRITICAL)